

ChargeMOD

Project Documentation
Study Reference Guide

1. Power BI Dashboard
2. Customer Retention & Churn Reduction
3. Maintenance Tab
4. Client AI Agent
5. Customer Support AI Agent
6. Dynamic Tariff System
7. Power BI HLD

June 2026

Power BI

Here's how you can explain it simply, stage by stage:

The One-Liner:

"I built a data pipeline that takes raw EV charging data from MongoDB, processes it overnight using AWS, and delivers a daily Power BI dashboard to the business team."

The Story (2-3 minutes):

The Problem:

"We had millions of EV charging transaction records sitting in MongoDB — session data, energy usage, revenue, charger status. But the business team had no easy way to see daily insights. They'd ask questions like 'how much energy did we deliver last month?' or 'which chargers are underperforming?' and someone would have to manually query the database. That's not scalable."

What I Built:

"I designed a nightly pipeline with four simple steps. Think of it like a conveyor belt:"

1. **Extract** — "Every night, an AWS Glue job picks up the latest transactions from MongoDB"
2. **Transform** — "It cleans the data — fixes timestamps, removes duplicates, joins charger details, calculates things like session duration and revenue — all using PySpark"
3. **Store** — "The clean data goes into S3 in Parquet format, which is like a super-compressed, column-based file format that saves storage cost and makes queries fast"
4. **Visualise** — "Power BI connects to that data through Athena (a serverless SQL engine), and the dashboard refreshes every morning automatically"

Why This Stack:

"Everything is serverless — Glue, Athena, S3 — so there's no server to manage, no cluster running 24/7. We pay only when the job runs. The whole pipeline costs under \$50 a month."

The Clever Parts (pick 2-3 to mention):

- "I used **Parquet + date partitioning** so Athena scans only the data it needs — reduced query cost by 95%"
- "The pipeline is **idempotent** — if it fails and reruns, it overwrites the same partition, so no duplicates"
- "MongoDB had **messy schemas** — same field could be string or integer. I wrote strict type casting with a quarantine table for bad records instead of crashing the whole job"
- "For **late-arriving data**, chargers sometimes go offline and upload data hours later. I added a 48-hour lookback buffer so late records get picked up in the next run"

The Result:

"The business team now opens Power BI every morning and sees yesterday's complete picture — energy delivered, revenue, charger utilisation, session trends — without asking engineering for anything."

That's it. Keep it conversational, don't over-explain AWS internals unless they ask, and always connect back to the business impact.

Customer Retention Project

Here's how you can explain it simply, broken into a natural flow:

Start with the problem (2 lines):

"We noticed that many of our DC charging customers were silently disappearing — their energy usage was dropping month after month, or they just stopped charging entirely. But nobody was reaching out to them before they left."

Then the solution (3-4 lines):

"So I built a system that automatically identifies these at-risk users every month. It compares each user's current month energy with their previous month. If someone dropped by more than 10 kWh, or went from active to inactive, they get flagged. The system then shows these users in a prioritised calling list inside our console, so our customer care team can call them, understand why they left, and try to bring them back."

Explain the smart part — clustering:

"We also segment users into Low, Medium, High, and Very High tiers based on their energy usage, using rolling percentiles so the thresholds adjust automatically as the business grows. We track each user's cluster for the last 5 months, so when an agent calls someone, they can see the full trajectory — like 'this person was Very High for 3 months and suddenly went Inactive', which tells a very different story than someone who was always Low."

The two outputs:

"The project has two faces — a Power BI dashboard for management to see retention trends, churn rates, and which segments are growing or shrinking. And a console page for the care team to actually do the calling — filter users, log call outcomes like 'Positive', 'Not Connected', or 'Negative', and track follow-ups."

End with impact:

"The whole point is simple — retaining an existing customer is 5 to 7 times cheaper than acquiring a new one. Every user we bring back is direct revenue recovered."

If they ask "what was the hardest part?":

"Handling messy real-world data — users don't charge every month, so you get gaps in their history. A user who skips one month isn't necessarily churned, they might just be sporadic. We had to carefully define what 'churned' actually means using the cluster transitions and energy differentials, not just simple month-to-month comparisons."

If they ask "what would you improve?":

"Right now the system is reactive — it catches users after they've already dropped. I'd add a predictive model that flags users likely to churn next month based on declining session frequency and shorter charging durations, so we can intervene before they leave, not after."

Keep it conversational, avoid jargon overload, and always connect back to **why** it matters for the business. That's what interviewers remember.

Maintenance Tab

Here's how you can explain it simply, broken into a natural flow:

Start with the problem:

"ChargeMod operates a network of EV charging stations across multiple locations. When a charger goes down, the support team had no quick way to know which chargers are down, how long they've been down, or what the last signals from the machine looked like. They had to manually dig through multiple databases to figure this out, which delayed repairs and hurt customer experience."

Then the solution:

"I built a Maintenance Tab in the ChargeMod Console that gives the support team a single screen showing three things: how many hours each charger was up or down yesterday, the last few machine logs for quick diagnosis, and status notifications highlighting errors. So instead of digging through raw data, they open the tab, filter by area, see what's down, check the logs, and decide whether to do a remote reset or send a technician."

Then the technical approach:

"Behind the scenes, it's a Python batch job that runs every night after midnight IST. It pulls charger metadata and machine logs from MongoDB, calculates downtime by looking at gaps between consecutive log timestamps — if a charger stays silent for more than 15 minutes, that gap counts as downtime. The results get saved back to MongoDB with a unique key per charger per day, so the job is safe to re-run without creating duplicates. The console just reads from this output collection."

Then one interesting challenge:

"One tricky problem was handling AC vs DC chargers — they store logs in completely different MongoDB collections with different schemas. DC chargers also need an extra mapping step to convert the charger ID to an internal logger ID before you can even fetch their logs. I handled this by building a clean pipeline where the data-fetching logic branches by charger type, but everything converges at the downtime calculator, which just works on sorted timestamps regardless of source."

End with impact:

"The main impact was reducing the time it takes to detect and fix charger outages. Before this, a charger could be down for hours before anyone noticed. Now the support team sees it first thing in the morning and can act immediately."

That's it — problem, solution, how it works, one challenge, and impact. Keep it under 2 minutes when speaking, and let the interviewer ask follow-up questions from there.

Client AI Agent Project

Here's how you can explain it simply and confidently in an interview:

Start with the problem (2 lines):

"EV charging station owners had to manually navigate complex dashboards to find things like energy usage, charger health, or how many users churned. It was slow, required training, and didn't support ad-hoc questions."

Then the solution (2-3 lines):

"We built a conversational AI agent — like a ChatGPT for their charging data. Owners just type questions in plain English like 'How much energy did my Technopark location use this month?' or 'Which chargers are down right now?' and get instant answers with charts."

How it works technically (keep it simple):

"The user's question goes to an LLM which figures out the *intent* — is this an energy question, a maintenance question, a user analytics question? Then it picks the right tool function from a registry of about 15-20 pre-built functions. Each function runs a fixed, parameterised SQL query on PostgreSQL and returns structured JSON. The LLM then formats that JSON into a nice response with text and charts. The key point is — the LLM never touches the database directly. It's just a smart router."

Why this design matters (show you understand tradeoffs):

"We didn't let the LLM write SQL because that's a security risk — it could bypass access controls, write DELETE queries, or expose other owners' data. Instead, every query is developer-written, reviewed, and tested. The owner's ID is injected from the session into every query automatically, so cross-account access is structurally impossible."

If they ask about scale/production readiness:

"We have Redis caching at the tool layer, nightly batch jobs for pre-aggregation so queries are fast, circuit breakers for LLM outages, rate limiting per owner, prompt regression tests that block deployment if accuracy drops below 95%, and full observability with trace IDs on every request."

If they ask about your role/team setup:

"It was a 3-week project, 2 sprints, 119 story points. We used modular monolith architecture so each developer could work on their own module — energy, maintenance, vehicle analytics — without merge conflicts, because each tool has its own prompt fragment file."

One-liner summary if they say "tell me in one line":

"It's a natural-language AI agent that lets EV charging station owners ask questions about their data in plain English and get instant answers with charts — built with LLM tool-calling, pre-written SQL, and strict access control."

Customer Support AI Agent

The One-Liner:

"I built an AI chat agent that helps support operators diagnose and fix EV charger problems just by typing a question — instead of checking 5-6 dashboards manually."

The 30-Second Version:

"ChargeMOD runs EV charging stations. When a charger stops working or a customer calls with a problem, our support team had to manually check logs, charger status, customer wallet, transactions — all in different dashboards. I built an AI agent where the operator just types something like 'CMOD-135 is not charging' and the system automatically pulls all the data, figures out the problem, and tells the operator what to do. It can also remotely start/stop chargers, book slots, and calculate charging time and cost."

The 2-Minute Technical Version:

"So the architecture is a modular monolith — one app, but cleanly split into modules internally. Let me walk you through what happens when an operator types a question."

There are **3 simple steps** inside:

Step 1 — Understand the question. The LLM reads the operator's message and decides which tool to call. Is this a charger problem? A customer lookup? A charging estimate? It picks the right tool and extracts the parameters — like the charger ID or phone number.

Step 2 — Fetch real data and think. Now the tool goes and calls our actual backend APIs — pulls charger logs, error codes, transaction history, live status — all in parallel to save time. Once the data comes back, a second LLM call looks at everything and decides: Can I answer from this data? Do I need to pull a troubleshooting guide? Or is the data unclear and I need to ask the operator more questions? We call these Path A, B, and C.

Step 3 — Generate the response. A third LLM call takes the decision and the data, and writes a clean response for the operator — with tables, follow-up suggestions, and action buttons if needed.

Now, a few important design choices:

The LLM never talks to the database. It only calls pre-built tool functions that run safe, fixed queries. So the AI can't accidentally delete data or bypass access control.

Anti-hallucination is critical because this agent can actually start and stop real chargers and shows real money balances. So we track every validated ID in a cache. If the LLM tries to make up a charger ID or phone number, it can't — it only has access to IDs that came from actual API responses.

For troubleshooting guides, we use a hybrid search — keyword search for exact error codes plus vector search for when operators describe problems in different words. A reranker picks the best match.

For charger name resolution — operators type messy things like 'yahoo supermarket' or 'cmod135'. We built a step-by-step matching system: try exact match first, then partial match, then fuzzy match, then location-based search. If it's not confident, it asks the operator to confirm before doing anything.

The whole thing streams responses in real-time using SSE, so operators see progress as it happens — not a loading spinner for 10 seconds."

Common Follow-up Answers (keep these ready):

"Why 3 LLM calls?"

"One call trying to do everything gets confused. Splitting it means each call has one job — pick the tool, analyse the data, write the response. Easier to debug, easier to test."

"Why modular monolith?"

"One small team, 3-week deadline. Microservices would add network calls on top of the 3 LLM calls we already make. But the code is modular inside — each module has its own folder and can be worked on independently."

"What was the hardest part?"

"Preventing hallucination. The AI can start real chargers and show real wallet balances — it can't make up data. We solved it with an entity cache that validates every ID through actual API calls and re-injects them into the prompt so the model can never invent fake data."

Dynamic Tariff Project

Here's how you can explain it simply, broken into a natural interview flow:

The One-Liner:

"I built a machine learning pipeline that predicts how busy each EV charger will be tomorrow, hour by hour, and automatically sets prices higher during peak hours and lower during off-peak hours — like surge pricing in Uber, but for EV charging."

The Problem (Why):

"Everyone wants to charge their EV at the same popular hours — say 6-9 PM. This overloads the grid and causes long wait times. We needed a way to spread out demand. The idea is simple: make busy hours slightly expensive and quiet hours cheaper, so some users naturally shift their charging to off-peak slots."

The Data:

"We pull 7 days of charging transaction history from MongoDB — how much energy each charger delivered, every hour. That's our raw input."

The ML Part (Two Levels):

"We forecast demand at two levels. First, network-wide — how busy is the entire charging network going to be each hour tomorrow? For this we tested XGBoost, Random Forest, and Prophet, and automatically pick whichever scores the lowest error. Second, per-charger — how busy will each individual charger be? This was trickier because most hours a charger is idle (zero energy). So I used a two-stage model: Stage 1 is a classifier that predicts 'will this charger be active this hour?', Stage 2 is a regressor trained only on active hours that predicts 'how much energy?'. We multiply both outputs together — so if the model is only 30% sure a charger will be active, the energy prediction gets scaled down to 30%. This avoids wild price swings for rarely-used chargers."

The Pricing Part:

"We combine the network score, the charger score, charger power capacity, and whether it's a weekend into one composite score between 0 and 1. Then we pass it through a sigmoid function to get a surge percentage — capped at +12% surcharge and -6% discount. Every charger is guaranteed at least 3 discount hours per day so there's always an incentive to charge off-peak."

Why These Choices:

"We used a batch pipeline (runs once at night) because prices only change daily — no need for real-time streaming. We used Random Forest because it handles the zero-inflated data well and is interpretable. We kept everything deterministic — fixed random seeds, sorted data, frozen model files — so any run can be exactly reproduced for audits. And each charger publishes independently to the API, so if one fails, the rest still go through."

One Key Thing That Makes It Smart:

"Each charger's price is based on its own demand pattern, not the fleet average. A small AC charger at a residential area and a 120kW DC fast charger on a highway have completely different usage curves. The per-charger normalisation and dynamic neutral point ensure each charger's pricing reflects its own reality."

That's roughly a 2-3 minute explanation. Keep it conversational — start with the problem, explain the approach simply, then drop in technical details only when the interviewer digs deeper.

"Before feeding data to the model, we had to handle some real-world data quality issues."

p95 Clipping:

"Sometimes a charger shows a crazy high energy reading — maybe a sensor bug or a one-time event. If we keep that, the model thinks it's normal and sets wild prices. So for each charger, we find its 95th percentile value and cap anything above it. Think of it like removing unrealistic outliers, but smartly — per charger, not globally."

Offline Detection:

"If a charger shows zero energy for 20+ hours in a day, it's probably switched off or broken — not that nobody came. We detect these days and remove them so the model doesn't wrongly learn 'this charger has no demand on certain days.'"

Log Transformation:

"Charging data is very skewed — most hours are small values, few hours are very high. Training directly on this makes the model chase big numbers and ignore the rest. So we apply $\log(1+x)$ to compress the range, train in log-space, then convert back with $\exp(x)$. Bonus: predictions always come out non-negative, which makes sense physically."

Imputation Strategy:

"New chargers or chargers with data gaps have no 'yesterday' value to use as a lag feature. Instead of putting zero (which would trick the model into thinking no one charges there), we fill it with the average of similar chargers from the same cluster for that hour. If even that's missing, we use the overall average for that hour. Zero is the absolute last resort. We also track which values were imputed with a flag column, so we know how much the prediction relied on real vs estimated data."

Clustering — Two Types:

Static Clustering (rule-based):

"We group chargers by type and power — like AC below 7.4 kW is AC_SLOW, DC 60-120 kW is DC_HIGH. This is just a simple rule, always available, even for a brand new charger with zero history."

Behavioural Clustering (KMeans):

"This is the interesting one. After a charger has at least 30 days of data, we look at how it actually behaves — not just what it is. We build a 36-feature profile for each charger:"

- "What's its hourly usage shape? Busy mornings or busy evenings?"
- "Is it more active on weekdays or weekends?"
- "How consistent is it — steady daily usage or unpredictable?"
- "What fraction of hours is it completely idle?"
- "What's its peak hour and how sharp is the peak?"

"Then we run KMeans on these profiles, testing k from 3 to 12 and picking the best k using silhouette score. What comes out is groups like — 'highway chargers that are busy 24/7', 'office chargers with a sharp 9 to 6

peak', 'mall chargers busy on weekends.' Two chargers can both be 60kW DC but land in completely different clusters because their usage patterns are different. These clusters help in two ways — they improve the model's predictions by adding a meaningful categorical feature, and they power the imputation fallback so new chargers get filled with values from chargers that behave similarly."